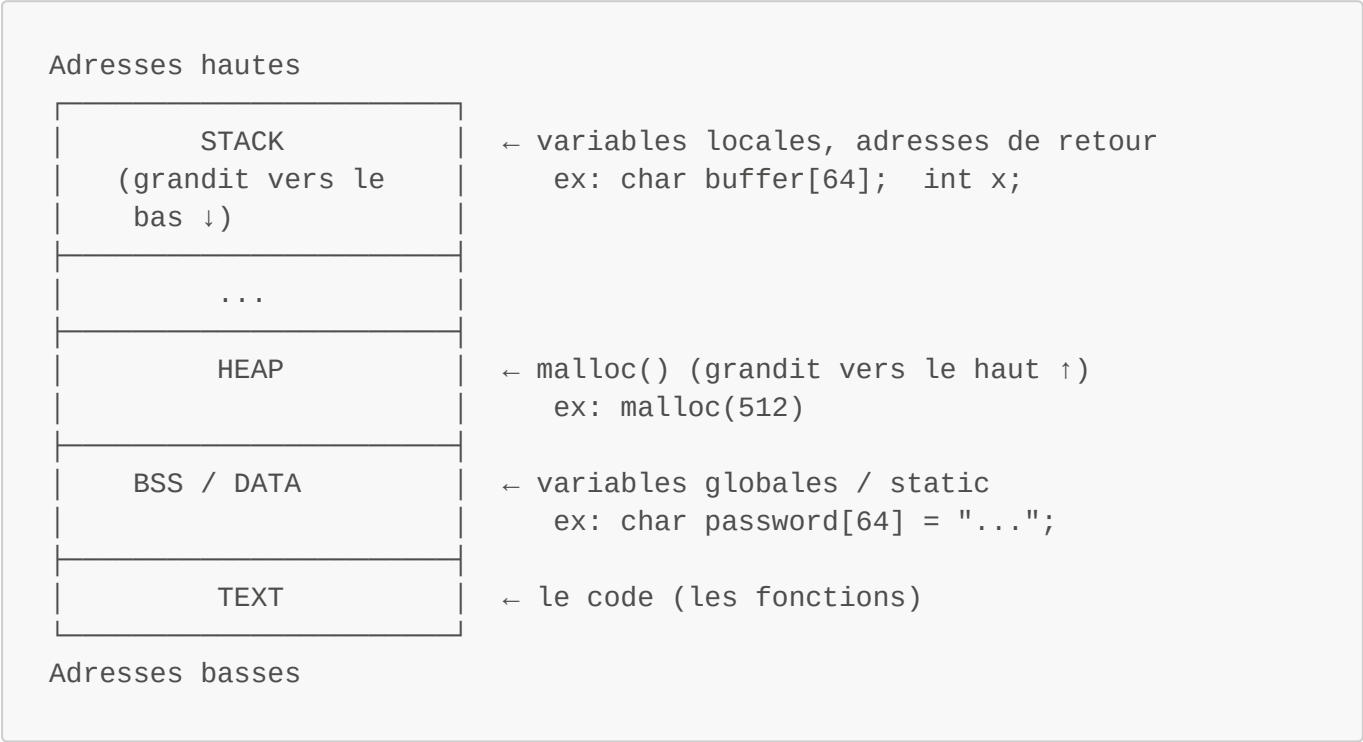


Révision C — Mémo autonome (reverse & exploitation)

1. Modèle mémoire — LE concept central

Tout le reverse tourne autour de **où vivent les données**.



Zone	Contenu	Durée de vie	Note reverse
Stack	locales, args, adresse de retour	jusqu'à sortie de fonction	rapide, taille limitée, cible des overflows
Heap	malloc/free	jusqu'au free	source de fuites mémoire (oubli de free)
Data/BSS	globales, static	tout le programme	adresse fixe → facile à inspecter en debug
Text	code des fonctions	tout le programme	adresses de fonctions (cibles d'un saut)

2. Pointeurs

Un pointeur = variable qui contient une **adresse mémoire**.

```
int x = 5; // x contient la valeur 5
int *p = &x; // p contient l'ADRESSE de x
```

```
printf("%d", *p); // *p déréférence → affiche 5
*p = 42;          // modifie x via le pointeur → x vaut 42
```

Symbole Sens

<code>&x</code>	adresse de <code>x</code>
<code>*p</code>	valeur pointée par <code>p</code> (déréférencement)
<code>int *p</code>	déclaration : <code>p</code> pointe vers un <code>int</code>

Mémoire dynamique

```
int *t = malloc(10 * sizeof(int)); // 10 int sur le HEAP, t = adresse du
1er
t[0] = 5;                          // t[0] == *(t+0)
free(t);                           // rend la mémoire au système
```

Pièges classiques :

- `int *t = malloc(...)` → `t` est un **pointeur**, pas un `int`. `t ≠ t[0]`.
- `t[0]` = la **valeur** ; `t` = l'**adresse**.
- Toujours `free()` ce qu'on `malloc()`, sinon fuite mémoire.
- `n++` retourne l'**ancienne** valeur puis incrémente ; `n + 1` retourne juste valeur+1.

Passage par valeur vs par adresse

```
void par_valeur(int n) { n = 42; } // modifie une COPIE → aucun effet
dehors
void par_adresse(int *p){ *p = 42; } // modifie l'ORIGINAL via
l'adresse

int a = 0;
par_valeur(a); // a vaut toujours 0
par_adresse(&a); // a vaut 42
```

En C, les arguments sont **copiés**. Pour modifier l'original, passer un pointeur.

3. Tableaux et chaînes de caractères

Une **chaîne** = tableau de **char** terminé par `\0` (octet nul).

```
char buffer[64]; // 64 octets, NON initialisés
char name[] = "abc"; // = {'a','b','c','\0'} → 4 octets
```

- `sizeof(buffer) = 64` (taille du tableau). Sur un **pointeur**, `sizeof` = taille du pointeur (8 octets en 64-bit), pas du tableau.
- Le nom d'un tableau « se dégrade » en pointeur vers son 1er élément quand on le passe à une fonction.
- **Aucune vérification de bornes en C.** Écrire dans `buffer[100]` alors que `buffer[64]` → dépassement. C'est la base des overflows.

Fonctions de chaînes

Fonction	Rôle	Danger
<code>strlen(s)</code>	longueur (sans <code>\0</code>)	lit jusqu'au <code>\0</code> ; si absent → lecture hors-limite
<code>strcmp(a, b)</code>	compare, <code>0</code> si égal	—
<code>strcpy(dst, src)</code>	copie chaîne	pas de limite → overflow
<code>strcat(dst, src)</code>	concatène	pas de limite → overflow
<code>memcpy(dst, src, n)</code>	copie <code>n</code> octets bruts	copie <code>n</code> même si <code>dst</code> trop petit → overflow
<code>gets(s)</code>	lit une ligne stdin	interdit : aucune limite, overflow garanti
<code>sprintf(buf, ...)</code>	écrit formaté dans <code>buf</code>	pas de limite → overflow
<code>scanf("%s", buf)</code>	lit un mot	pas de limite → overflow
<code>fgets(buf, size, f)</code>	lit <code>size-1</code> max	sûr (borné)
<code>snprintf(buf, size, ...)</code>	formaté borné	sûr

Règle : `strcpy`, `strcat`, `sprintf`, `gets`, `scanf("%s")` = **non bornés** → dangereux. Versions sûres : `strncpy`, `strncat`, `snprintf`, `fgets`.

4. Le buffer overflow

```
void vulnerable(char *input, size_t len) {
    char buffer[64];
    memcpy(buffer, input, len);    // si len > 64 → on écrase la pile !
}
```

Disposition de la pile quand `buffer` déborde :

```
[ buffer[64] ][ RBP sauvé ][ adresse de retour ]
^écriture part d'ici -----> déborde ici
```

- `buffer` fait 64 octets. Copier plus écrase d'abord le **RBP sauvé** puis l'**adresse de retour**.
- Contrôler l'adresse de retour = **contrôler RIP** = rediriger l'exécution vers une adresse choisie (ex : une fonction cachée qui lance un shell).

- `%p` affiche une **adresse** en hexa → sert à repérer les cibles (buffer, fonctions).

Notions liées :

- `size_t` = entier non signé pour des tailles/longueurs.
- Entrée du programme : `argv[1]` (ligne de commande) ou `stdin` (`fread/fgets`).

```
int main(int argc, char **argv) {    // argc = nb d'args, argv[] = tableau
de chaînes
    // argv[0] = nom du programme, argv[1] = 1er argument utilisateur
}
```

5. Fonctions, prototypes, portée

```
int check(void);                // prototype : signature déclarée au compilateur
...
int check(void) {                // définition : le vrai code
    return 0;
}
```

- **Variable globale** (déclarée hors fonction) : visible partout, **adresse fixe** → idéale à inspecter/modifier en debug.
- **Variable locale** : sur la pile, existe seulement dans la fonction → à modifier au runtime quand un breakpoint est actif.
- Une fonction **jamais appelée** existe quand même dans le segment TEXT → on peut sauter dessus en debug/exploit.
- `while(1) { ... }` = boucle infinie ; `sleep(n)` = pause (laisse le temps d'attacher un debugger).

6. Structures (`struct`)

```
typedef struct {
    char username[50];
    char password[50];
    int is_admin;
} User;

User u;                // une structure
u.is_admin = 1;        // accès à un champ avec le point
User *p = &u;
p->is_admin = 1;        // via un pointeur : flèche
```

- Une `struct` regroupe des champs **contigus** en mémoire → dans un dump, les champs se suivent.
- Accès : `variable.champ` (direct) ou `pointeur->champ` (via pointeur).

- Des secrets stockés en globales/`struct` restent **en clair en mémoire** → extractibles par un dump.

Notion sécurité (CISSP) : *data at rest in memory*. Un secret en RAM, même jamais affiché, est récupérable via un dump du processus. Défense : minimiser sa durée de vie, écraser (`memset(p, 0, taille)`) après usage.

7. Entrées / sorties courantes

```
printf("%d %s %p %c\n", entier, chaine, pointeur, caractere);
scanf("%d", &choice); // & obligatoire : scanf veut une
ADRESSE
fgets(buf, sizeof(buf), stdin); // lecture bornée (sûr)
buf[strcspn(buf, "\n")] = 0; // enlève le \n final laissé par
fgets
fflush(stdout); // force l'affichage immédiat
```

Spécificateurs `printf` utiles en reverse :

%d	entier décimal	%x	hexa	%p	adresse/pointeur
%s	chaîne	%c	caractère	%u	non signé

8. Compilation — flags fréquents en reverse/exploit

```
gcc -g -fno-pie -no-pie -o prog prog.c
```

Flag	Effet	Utilité
-g	symboles de debug	GDB voit noms de variables/lignes
-fno-pie -no-pie	désactive PIE	adresses fixes → exploit reproductible
-fno-stack-protector	pas de <i>canary</i>	overflow non détecté
-z execstack	pile exécutable	shellcode sur la pile possible
-O0	pas d'optimisation	code lisible, proche du source

Protections modernes (souvent désactivées en TP) : **ASLR** (adresses aléatoires), **stack canary** (valeur sentinelle avant l'adresse de retour), **NX/DEP** (pile non exécutable), **PIE** (binaire relocalisable).

9. Checklist express

- ☐ Distinguer **valeur** (`x`, `t[0]`, `*p`) vs **adresse** (`&x`, `p`, nom de tableau).
- ☐ `malloc` → HEAP (il faut `free`). Locales → STACK. Globales → DATA/BSS.
- ☐ Chaîne = tableau de `char` finissant par `\0`.

- ☐ Fonctions dangereuses : `gets`, `strcpy`, `strcat`, `sprintf`, `memcpy` sans borne, `scanf("%s")`.
 - ☐ Overflow = écrire au-delà du buffer → écraser RBP puis adresse de retour → contrôler RIP.
 - ☐ `%p` = lire une adresse ; sert à repérer les cibles.
 - ☐ Globale = adresse fixe = facile à inspecter/modifier en debugger.
 - ☐ `struct` = champs contigus ; secrets en mémoire = extractibles par dump.
 - ☐ Flags : `-g` (debug), `-no-pie/-fno-stack-protector` (exploit reproductible).
-

10. Mini-quiz (réponses repliées)

1. Différence entre `t` et `t[0]` si `int *t = malloc(...)` ?
2. Pourquoi `memcpy(buf, src, len)` peut-il être dangereux ?
3. Où vit une variable `char password[64]` déclarée hors de toute fonction, et pourquoi c'est pratique en GDB ?
4. Que retourne `strcmp(a, b)` si `a` et `b` sont identiques ?
5. Pourquoi `gets()` est-il pire que `fgets()` ?
6. Que contrôle-t-on en écrasant l'adresse de retour sur la pile ?

► Réponses